

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 9

Виконав:

студент групи кн-25-1

Рафієв Б. С.

Перевірив:

доцент кафедри КІЕ

Сидоренко В.М.

Кременчук 2026

Тема. **Стиснення даних. Кодування і декодування Гафмена**

Мета: набути практичних навичок застосування алгоритму оптимального кодування Гафмена.

ХІД РОБОТИ

Завдання для самостійного розв’язання:

Умова **задачі:** Маємо вхідний текст
AABABVABACVCAADEEAABABABVBCVCAADEE. Закодувати текст,
використовуючи алгоритм Гафмена. Побудувати двійкове дерево. Оцінити ефект від кодування порівняно з неоптимальним випадком.

1. Візуальна структура та аналіз частот символів

Для побудови оптимального коду спершу виконаємо частотний аналіз заданого рядка. Загальна довжина послідовності становить $N = 35$ символів.

Порахуємо абсолютну частоту появи кожного унікального символу (кількість входжень) та обчислимо їхні ймовірності:

A	15	15/35
B	10	10/35
C	4	4/35
E	4	4/35
D	2	2/35

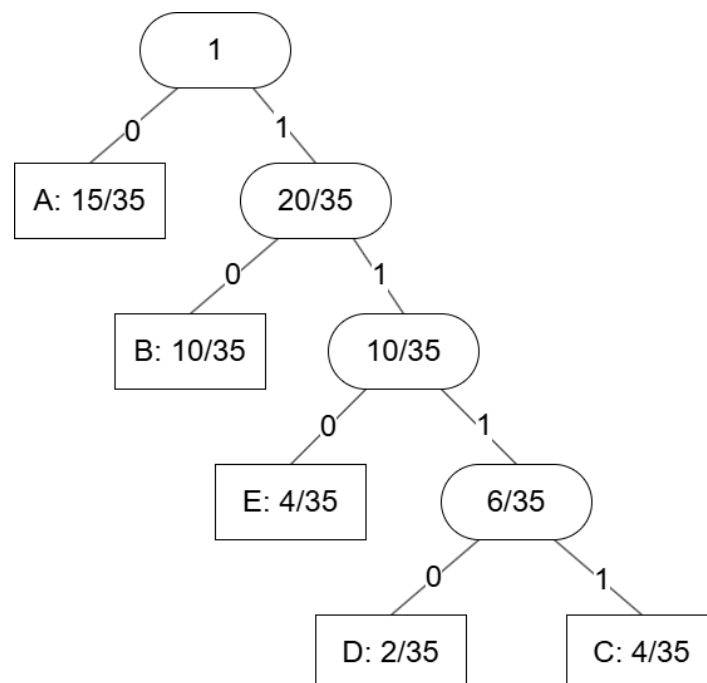
Топологічна особливість структури: Символи A та B сумарно займають понад 71% усього тексту ($15 + 10 = 25$ символів із 35). Відповідно до жадібної стратегії Гафмена, ці символи повинні отримати найкоротші кодові комбінації для досягнення максимального стиснення.

2. Покроковий опис розв'язання задачі (Побудова дерева Гафмена)

Алгоритм реалізує жадібний підхід: на кожному кроці обираються два вузли з найменшою сумарною вагою й об'єднуються в один батьківський вузол. Лівій гілці присвоюємо значення 0, правій – 1.

- Крок 1. Формуємо початковий список листів, відсортований за зростанням ймовірностей: [D: 2/35], [C: 4/35], [E: 4/35], [B: 10/35], [A: 15/35]
- Крок 2. Об'єднуємо два найменші вузли D:2/35 та C:4/35. Отримуємо внутрішній вузол (D,C): 6. Новий список: [E: 4/35], [(D,C): 6/35], [B: 10/35], [A: 15/35]
- Крок 3. Об'єднуємо два найменші вузли E:4/35 та (D,C):6/35. Отримуємо внутрішній вузол (E,(D,C)): 10/35. Новий список: [B: 10/35], [(E,(D,C)): 10/35], [A: 15/35]
- Крок 4. Об'єднуємо два найменші вузли B:10/35 та (E,(D,C)):10/35. Отримуємо внутрішній вузол (B,(E,(D,C))): 20/35. Новий список: [A: 15/35], [(B,(E,(D,C))): 20/35]
- Крок 5. Фінальне об'єднання двох останніх вузлів A:15/35 та (B,(E,(D,C))):20/35. Отримуємо корінь дерева із загальною вагою 1.

Графічне представлення отриманого двійкового дерева Гафмена:



Шляхом обходу дерева від кореня до листів отримуємо такі префіксні коди для кожного символу:

$A \rightarrow 0$ (довжина: 1 біт)

$B \rightarrow 10$ (довжина: 2 біти)

$E \rightarrow 110$ (довжина: 3 біти)

$D \rightarrow 1110$ (довжина: 4 біти)

$C \rightarrow 1111$ (довжина: 4 біти)

Оскільки жоден код не є префіксом іншого, дані можуть бути однозначно декодовані без використання розділових знаків.

3. Математичне обґрунтування та оцінка ефекту від стиснення

Для оцінки ефективності алгоритму порівнюємо отриманий об'єм стиснених даних із неоптимальними варіантами зберігання інформації.

1. Об'єм даних за алгоритмом Гафмена (V_{Huffman}):

Обчислюється як сума добутків частоти кожного символу на довжину його бітового коду:

$$V_{\text{Huffman}} = (15 * 1) + (10 * 2) + (4 * 3) + (2 * 4) + (4 * 4)$$

$$V_{\text{Huffman}} = 15 + 20 + 12 + 8 + 16 = 71 \text{ біт}$$

Середня довжина коду на один символ становить: $71 / 35$ біта.

2. Об'єм даних при рівномірному неоптимальному кодуванні (V_{Uniform}):

Для кодування 5 унікальних символів (A, B, C, D, E) мінімально необхідна фіксована довжина коду становить $\log_2 5 = 3$ біти на один символ (оскільки 2 біти дають лише 4 комбінації, чого недостатньо). $V_{\text{Uniform}} = 35 \text{ символів} * 3 \text{ біт} = 105 \text{ біт}$

3. Об'єм даних у стандартному байтовому представленні (V_{ASCII}):

У звичайних текстових файлах без стиснення (ASCII, UTF-8) під кожен символ відводиться рівно 1 байт (8 бітів). $V_{\text{ASCII}} = 35 \text{ символів} * 8 \text{ бітів} = 280 \text{ біт}$

Порівняльний аналіз результатів стиснення:

1. Порівняно з рівномірним кодуванням (3 біти):

$$\text{Ефект стиснення: } \frac{105 - 71}{105} * 100\% = 32.38.$$

Висновок: Економію отримано завдяки виділенню найчастішому символу А лише 1 біта замість

2. Порівняно зі стандартним ASCII-кодуванням (8 біт):

$$\text{Ефект стиснення: } \frac{280 - 71}{208} * 100\% = 74.64\%$$

Висновок: Об'єм файлу зменшився майже в 4 рази.

Висновки: Під час виконання цієї практичної роботи я набув практичних навичок роботи з алгоритмами стиснення даних без утрат та детально дослідив жадібну стратегію на прикладі оптимального префіксного кодування Гафмена.

На прикладі заданого текстового рядка було математично доведено високу ефективність алгоритму за умов нерівномірного розподілу частот символів. Алгоритм Гафмена дозволив скоротити об'єм даних до 71 біта, що на 32.38% краще за мінімальне фіксоване кодування та на 74.64% ефективніше за стандартне побайтове збереження тексту.

Проведений аналіз демонструє класичну властивість жадібного вибору: мінімізація довжини коду для найбільш частотних елементів на кожному кроці гарантує отримання глобально мінімального розміру результуючої бітової послідовності.

Відповіді на контрольні запитання

1. Що таке кодування Гафмена та як воно працює? Кодування Гафмена – це класичний алгоритм стиснення даних без утрат, який оптимізує розмір інформації шляхом заміни стандартних символів на префіксні коди змінної довжини. В основі його роботи лежить аналіз частоти появи кожного знака у вихідному тексті: символам, які зустрічаються найчастіше, присвоюються найкоротші бітові комбінації, тоді як рідкісні літери отримують довші коди. Завдяки такому нерівномірному розподілу загальна довжина кінцевої бітової послідовності суттєво зменшується, що дозволяє значно економити дисковий простір або час на передачу даних по мережі.

2. Як визначається оптимальний двійковий код Гафмена для стиснення даних? Оптимальний двійковий код визначається шляхом покрокової побудови бінарного дерева за жадібною стратегією, де рух відбувається знизу догори. Спочатку всі унікальні символи сортуються за зростанням їхніх частот або ймовірностей, після чого два найменших вузли вилучаються і об'єднуються в один батьківський вузол із вагою, що дорівнює їхній сумі. Цей процес повторюється циклічно до створення єдиного кореня дерева. Потім усім лівим гілкам дерева присвоюється логічний нуль, а правим – одиниця, завдяки чому фінальний унікальний код кожної літери формується як послідовний шлях від самого кореня до відповідного листа.

3. Які переваги має кодування Гафмена над іншими методами стиснення даних? Головною перевагою кодування Гафмена є його доведена математична оптимальність серед усіх методів посимвольного стиснення інформації з нерівномірним розподілом частот. Отримані коди мають властивість безпрефіксності, тобто кодова комбінація одного символу ніколи не є початком іншого, що дозволяє комп'ютеру миттєво розпізнавати літери у суцільному бітовому потоці без використання додаткових маркерів чи розділових знаків. Крім того, алгоритм є обчислювально ефективним, простим для розуміння і легко реалізується на практиці за допомогою стандартних черг із пріоритетом.

4. Як відбувається декодування даних, закодованих за допомогою кодування Гафмена? Процес декодування даних полягає в послідовному зчитуванні стисненого бітового потоку зліва направо та одночасному обході раніше побудованого дерева Гафмена, починаючи з його кореня. Зустрічаючи біт «0», алгоритм переходить до лівого нащадка поточного вузла, а зустрічаючи «1» – до правого. Цей рух за вказаними координатами триває доти, доки алгоритм не досягне кінцевого листа дерева, який містить розшифрований символ. Щойно символ виділено, він записується у відновлений текст, а вказівник декодера миттєво повертається на корінь дерева для обробки наступних бітів.

5. Які є можливі недоліки кодування Гафмена? Основним недоліком алгоритму є необхідність обов'язкового збереження структури кодового дерева або таблиці частот разом із самим стисненим файлом, оскільки без цієї інформації декодування стає неможливим, а для малих файлів такі службові дані можуть навіть перевищити ефект від стиснення. Також класичний метод є двопрхідним, адже вимагає спочатку повністю прочитати весь об'єм даних для підрахунку частот, і лише потім почати кодування, що обмежує його застосування в системах передачі потокового відео чи аудіо в реальному часі. Окрім цього, посимвольний підхід не враховує контекстні зв'язки між сусідніми літерами, через що він програє сучаснішим словниковим алгоритмам типу LZ77.

6. Для чого використовується побудова дерева в кодуванні Гафмена? Побудова бінарного дерева в алгоритмі Гафмена використовується як фундаментальний каркас для правильного розподілу та генерації унікальних кодових комбінацій. Воно виконує роль математичного інструменту, який на етапі кодування гарантує безпрефіксність і оптимальність довжини бітових послідовностей відповідно до ваги кожного символу. Під час зворотного процесу дерево слугує чіткою та однозначною «дорожньою картою» для декодера, яка дозволяє комп'ютеру безпомилково розбивати суцільну стрічку з нулів та одиниць назад на окремі літери без використання штучних пробілів чи роздільників.